

GRASP metaheuristika za dvostepeni transportni problem sa fiksnim troškovima (TS-FCTP)

Mateja Stojanović
Broj indeksa: mi251107

2. septembar 2026.

Sažetak

U ovom radu razmatra se Dvostepeni transportni problem sa fiksnim troškovima (engl. *Two-Stage Fixed-Charge Transportation Problem*, TS-FCTP), problem kombinatorne optimizacije koji modeluje distribuciju robe kroz dvostepeni lanac snabdevanja – od dobavljača do distributivnih centara i od distributivnih centara do kupaca – uz varijabilne i fiksne troškove transporta na svakoj vezi. Predložena je GRASP metaheuristika (engl. *Greedy Randomized Adaptive Search Procedure*) koja kombinuje adaptivno podešavan parametar α , lokalnu pretragu tipa VND sa pet okolina, egzaktno filtriranje kandidata zasnovano na zatvorenoj formuli za trošak druge faze, skup elitnih rešenja sa raznolikošću po Jaccardovom rastojanju, prekranje putanja (engl. *path relinking*), ILS perturbaciju elitnih rešenja i ruin-and-recreate perturbaciju pri dugotrajnom zastoju. Implementacija je vektorizovana (NumPy) i paralelizovana (**multiprocessing**) na nivou nezavisnih iteracija, čime je omogućeno višestruko izvršavanje algoritma po instanci i dobijanje statistički smislenih pokazatelja kvaliteta. Algoritam je testiran na skupu instanci malih, srednjih i velikih dimenzija i upoređen sa rešenjima dobijenim CPLEX solverom.

Ključne reči: dvostepeni transportni problem sa fiksnim troškovima, GRASP, metaheuristika, egzaktno filtriranje kandidata, okolina, skup elitnih rešenja, path relinking, perturbacija, vektorizacija, paralelizacija, kombinatorna optimizacija.

1 Opis problema i matematička formulacija

Oznake i ulazni podaci. Posmatra se dvostepeni lanac snabdevanja koji se sastoji od tri sloja čvorova povezanih u dve etape transporta: $i \rightarrow j \rightarrow k$, gde je $i \in I$ skup dobavljača (engl. *suppliers*), $j \in J$ skup distributivnih centara (u nastavku DC) i $k \in K$ skup kupaca (engl. *customers*). Ulazni podaci problema su:

- s_i – raspoloživa ponuda dobavljača $i \in I$;
- d_k – potražnja kupca $k \in K$;
- b_{ij} – varijabilni trošak transporta od dobavljača i do DC-a j ;

- f_{ij} – fiksni trošak aktiviranja veze (i, j) ;
- c_{jk} – varijabilni trošak transporta od DC-a j do kupca k ;
- g_{jk} – fiksni trošak aktiviranja veze (j, k) .

Ukupna ponuda jednaka je ukupnoj potražnji, tj. važi $\sum_{i \in I} s_i = \sum_{k \in K} d_k$, pa problem uvek ima dopustivo rešenje.

Opis problema. Dvostepeni transportni problem sa fiksnim troškovima (engl. *Two-Stage Fixed-Charge Transportation Problem*, TS-FCTP) problem je kombinatorne optimizacije koji je prvi put uveden u radu Jawahar i Balaji [?]. Za svaku vezu u mreži definisana su dva troška: varijabilni, proporcionalan količini koja se njome prevozi, i fiksni, koji se plaća jednom ako se veza uopšte koristi, tj. ako je protok na njoj pozitivan. Cilj je minimizacija ukupnog troška transporta uz poštovanje ograničenja kapaciteta dobavljača i tačno ispunjenje potražnje svih kupaca. Odluka o tome koje veze će biti aktivirane (koji DC-ovi i koje veze će se koristiti) ima kombinatornu prirodu i čini problem suštinski težim od klasičnog transportnog problema, u kome se aktiviranje veza ne naplaćuje.

Oblasti primene. TS-FCTP se javlja u različitim oblastima logističkog planiranja:

- **upravljanje lancima snabdevanja** – planiranje toka robe od fabrika do magacina (distributivnih centara) i dalje do maloprodajnih objekata;
- **distribucija energenata** – alokacija proizvoda od proizvodnih jedinica do skladišnih (distributivnih) stanica i do potrošača;
- **telekomunikacione mreže** – rutiranje podataka kroz čvorove mreže uz fiksne troškove aktivacije veza;
- **humanitarna logistika** – distribucija pomoći od centralnih magacina do regionalnih centara i do krajnjih korisnika.

Promenljive. U matematičkoj formulaciji koriste se sledeće promenljive:

- $x_{ij} \geq 0$ – količina koja se transportuje od dobavljača i do DC-a j (tok I. etape);
- $y_{jk} \geq 0$ – količina koja se transportuje od DC-a j do kupca k (tok II. etape);
- $z_{ij} \in \{0, 1\}$ – indikator da li se koristi veza (i, j) ;
- $w_{jk} \in \{0, 1\}$ – indikator da li se koristi veza (j, k) .

Matematička formulacija. Formulacija je preuzeta iz rada Calvete, Galé, Iranzo i Toth [?], koji su, za razliku od prvobitne formulacije [?], eksplicitno uveli i ograničenje balansa protoka u distributivnim centrima. Funkcija cilja glasi:

$$\min \sum_{i \in I} \sum_{j \in J} (b_{ij} x_{ij} + f_{ij} z_{ij}) + \sum_{j \in J} \sum_{k \in K} (c_{jk} y_{jk} + g_{jk} w_{jk}), \quad (1)$$

uz ograničenja:

$$\sum_{j \in J} x_{ij} \leq s_i, \quad \forall i \in I, \quad (2)$$

$$\sum_{j \in J} y_{jk} = d_k, \quad \forall k \in K, \quad (3)$$

$$\sum_{i \in I} x_{ij} - \sum_{k \in K} y_{jk} = 0, \quad \forall j \in J, \quad (4)$$

$$x_{ij} \leq s_i z_{ij}, \quad \forall i \in I, \forall j \in J, \quad (5)$$

$$y_{jk} \leq d_k w_{jk}, \quad \forall j \in J, \forall k \in K. \quad (6)$$

Značenje funkcije cilja i ograničenja. Funkcija cilja (??) minimizuje ukupni trošak distribucije: sabirci $b_{ij}x_{ij}$ i $c_{jk}y_{jk}$ su varijabilni troškovi, proporcionalni protoku, dok su $f_{ij}z_{ij}$ i $g_{jk}w_{jk}$ fiksni troškovi koji se plaćaju jednom ako se odgovarajuća veza koristi. Ograničenje (??) obezbeđuje da nijedan dobavljač ne šalje više robe nego što ima na raspolaganju. Ograničenje (??) garantuje da se potražnja svakog kupca tačno zadovolji. Ograničenje (??) predstavlja balans protoka u distributivnim centrima: DC ne skladišti niti stvara robu, već je samo prosleđuje, pa je ukupni ulaz u svaki DC jednak njegovom ukupnom izlazu. Ograničenja (??) i (??) su Big-M ograničenja koja povezuju kontinualne tokove sa binarnim indikatorima: ako je protok na vezi pozitivan, odgovarajući indikator mora biti jednak 1, čime se modeluje fiksni trošak aktivacije veze.

Složenost problema. Problem TS-FCTP pripada klasi NP-teških problema [?]: prisustvo fiksnih troškova unosi diskontinuitete u funkciju cilja i pretvaranje problema u problem mešovitog celobrojnog programiranja, pa egzaktno rešavanje ne može da prati rast dimenzija realnih instanci.

Postojeći načini rešavanja. U literaturi se za TS-FCTP javljaju sledeći pristupi:

- **egzaktne metode** – rešavanje MIP formulacije komercijalnim solverima (npr. IBM ILOG CPLEX) uz primenu *branch-and-bound* i *branch-and-cut* algoritama; za male instance postiže se optimalnost, dok se za srednje i velike instance vreme rešavanja meri satima [?];
- **genetski algoritmi** – prvi predloženi za TS-FCTP u radu Jawahar i Balaji [?], a zatim razvijani u radu Raj i Rajendran [?];
- **simulirano kaljenje** – predloženo za TS-FCTP u radu Balaji i Jawahar [?];
- **hibridni genetski algoritmi i matheuristike** – kombinuju evolucijsku pretragu sa egzaktnim rešavanjem potproblema ili sa snažnom lokalnom pretragom [?, ?].

U ovom radu predložena je GRASP metaheuristika koja pohlepno-nasumičnu konstrukciju kombinuje sa promenljivom lokalnom pretragom (VND), skupom elitnih rešenja, prekranjavanjem putanja i perturbacijom; detaljan opis dat je u narednoj sekciji.

2 Metaheuristika GRASP i njena adaptacija

2.1 Osnove GRASP-a

GRASP (*Greedy Randomized Adaptive Search Procedure*) je iterativna metaheuristika koju su uveli Feo i Resende [?]. Svaka iteracija se sastoji od dve faze:

1. **Faza konstrukcije:** gradi se dopustivo rešenje pohlepno-nasumičnim pristupom. U svakom koraku formira se *Restricted Candidate List* (RCL) – skup kandidata čija je ocena u opsegu $[\min, \min + \alpha(\max - \min)]$ – i nasumično se bira jedan element RCL-a. Parametar $\alpha \in [0, 1]$ kontroliše stepen nasumičnosti: $\alpha = 0$ odgovara čistoj pohlepnoj heuristici, a $\alpha = 1$ nasumičnom izboru.
2. **Faza lokalne pretrage:** dobijeno rešenje se poboljšava lokalnom pretragom u skupu okolnih rešenja.

Algoritam se ponavlja unapred zadati broj iteracija i pamti globalno najbolje pronađeno rešenje. Osnovni pseudokod GRASP-a dat je Algoritmom ??.

Algoritam 1: Osnovni GRASP(T, α)

```

1  $sol^* \leftarrow \emptyset$ ;  $best\_cost \leftarrow +\infty$ ;
2 for  $it = 0$  to  $T - 1$  do
3    $sol \leftarrow GreedyRandomizedConstruction(\alpha)$ ;
4    $sol \leftarrow LocalSearch(sol)$ ;
5   if  $sol.cost < best\_cost$  then
6      $sol^* \leftarrow sol$ ;  $best\_cost \leftarrow sol.cost$ ;
7   end
8 end
9 return  $sol^*$ 
```

2.2 Pojmovnik

Radi preglednosti, u nastavku su objašnjeni skraćeni i kolokvijalni nazivi koji se koriste u opisu implementacije:

- **DC** – distributivni centar (čvor srednjeg sloja J).
- **I. etapa** – transportna etapa dobavljač $\rightarrow$ DC (tokovi x , troškovi b, f).
- **II. etapa** – transportna etapa DC $\rightarrow$ kupac (tokovi y , troškovi c, g).
- **luk** – veza između dva čvora susednih slojeva kojom teče pozitivan tok, npr. luk (j, k) znači $y_{jk} > 0$.
- **otvoren DC** – DC kroz koji trenutno prolazi bar jedan pozitivan tok (skup `open_dc`).
- **RCL** (*Restricted Candidate List*) – skup kandidata u fazi konstrukcije čija je ocena unutar α -opsega od najbolje.
- **VND** (*Variable Neighborhood Descent*) – lokalna pretraga koja redom isprobava više okolina dok se ne dostigne lokalni optimum ni u jednoj.
- **first-improvement** – pravilo prihvatanja prvog pronađenog pomaka koji smanjuje trošak, bez pretrage cele okoline.
- **skup elitnih rešenja** (*elite pool*) – kolekcija najboljih i međusobno raznolikih rešenja pronađenih tokom pretrage, izvor za prekrajanje putanja i ILS.
- **prekrajanje putanja** (*path relinking*) – postupak koji kombinuje dva elitna rešenja tako što jedno postepeno transformiše u drugo, razmenjujući kolone kupaca.
- **ILS** (*Iterated Local Search*) – perturbacija elitnog rešenja praćena ponovnom lokalnom pretragom.

- **ruin-and-recreate perturbacija** (`_shake` u kodu) – perturbacija koja se aktivira pri dugotrajnom zastoju: zatvara skupo polovinu otvorenih DC-ova i ponovo raspoređuje njihov tok.
- **batch** – grupa od n_{workers} zadataka (restart, ILS ili prekrajanje putanja) koja se u paralelnoj verziji izvršava zajedno u radničkim procesima.

2.3 Implementacija

Implementacija (folder `GRASP_part_parallel/` u repozitorijumu projekta) sastoji se iz tri fajla: `utils.py` (učitavanje instance, infrastruktura za multi-run eksperimente i izveštaj), `GRASP.py` (ceo algoritam) i `main.py` (CLI ulaz). U nastavku je detaljno opisana svaka komponenta iz `GRASP.py`, a potom kompletan pseudokod algoritma.

2.3.1 Reprezentacija rešenja – klasa `Solution`

Rešenje se predstavlja parom matrica toka i skupom otvorenih DC-ova:

- $x \in \mathbb{Z}_{\geq 0}^{I \times J}$ – tokovi I. etape (dobavljač $\rightarrow$ DC);
- $y \in \mathbb{Z}_{\geq 0}^{J \times K}$ – tokovi II. etape (DC $\rightarrow$ kupac);
- $\text{open_dc} \subseteq \{1, \dots, J\}$ – skup otvorenih DC-ova (DC j je otvoren ako $\sum_i x_{ij} > 0$ ili $\sum_k y_{jk} > 0$).

Ukupni trošak rešenja računa metoda `compute_cost()` kao sumu varijabilnih i fiksnih troškova u obe etape:

$$\text{cost}(x, y) = \sum_{i \in I} \sum_{j \in J} b_{ij} x_{ij} + \sum_{j \in J} \sum_{k \in K} c_{jk} y_{jk} + \sum_{i \in I} \sum_{j \in J} f_{ij} [x_{ij} > 0] + \sum_{j \in J} \sum_{k \in K} g_{jk} [y_{jk} > 0], \quad (7)$$

gde je $[P]$ Iversonov simbol: $[P] = 1$ ako je tvrdjenje P tačno, a 0 u suprotnom. Fiksni trošak veze se dakle plaća tačno kada je protok na njoj pozitivan. Svaki poziv `eval_cost()` – stvarno izračunavanje vrednosti funkcije cilja – uvećava globalni brojač `eval` (broj poziva funkcije cilja). Metoda `compute_cost()` je omotač koji poziva `eval_cost()` i zatim osvežava `open_dc`; u `eval` se ne beleži broj poziva `compute_cost()`, jer lokalna pretraga u prihvatnoj probi poziva `eval_cost()` i bez osvežavanja `open_dc`. Metoda `is_valid()` proverava dopustivost rešenja: potražnja svakog kupca, kapacitet svakog dobavljača i balans protoka u svakom DC-u.

Ključna arhitekturna odluka: tokom pretrage se direktno modifikuju isključivo y -tokovi (II. etapa); x -tokovi (I. etapa) se pri svakoj promeni y -toka iznova rekonstruišu pohlepnom procedurom REPAIR. Ovaj pristup smanjuje dimenzionalnost prostora pretrage jer algoritam nikad eksplicitno ne upravlja x -promenljivama.

2.3.2 Procedura REPAIR

Za svaki DC j sa pozitivnim ukupnim prilivom ($\text{demand}_j = \sum_k y_{jk}$), dobavljači se rangiraju po celobrojnoj oceni koja kombinuje varijabilni i amortizovani fiksni trošak,

$$\text{score}_i = b_{ij} \cdot \text{demand}_j + f_{ij}, \quad (8)$$

a zatim se tokovi pohlepno dodeljuju u tom redosledu dok se zahtev DC-a ne zadovolji (pri izjednačenim ocenama prednost ima dobavljač sa najvećim preostalim kapacitetom). Ovo

je celobrojna varijanta ocene $b_{ij} + f_{ij}/\text{demand}_j$ – pošto je $\text{demand}_j > 0$ fiksno za dati DC, isto rangiranje se dobija skaliranjem sa demand_j , čime se izbegava deljenje u pokretnom zarezu i gubitak preciznosti. Ako neki DC ne može biti u potpunosti snabdeven (nedostatak ukupnog kapaciteta), procedura vraća neuspeh. Pseudokod je dat Algoritmom ??.

Algoritam 2: Repair(sol, D)

```

1   $sol.x \leftarrow 0$ ;
2   $rem\_s \leftarrow D.s.copy()$ ;
3  for  $j = 1$  to  $J$  do
4       $\text{demand}_j \leftarrow \sum_k y_{jk}$ ;
5      if  $\text{demand}_j = 0$  then continue;
6      Sortiraj dobavljače po  $\text{score}_i = b_{ij} \cdot \text{demand}_j + f_{ij}$ ;
7       $rem \leftarrow \text{demand}_j$ ;
8      for svakog  $i$  u sortiranom redosledu do
9           $q \leftarrow \min(\text{rem\_s}[i], rem)$ ;
10          $x_{ij} += q$ ;  $\text{rem\_s}[i] -= q$ ;  $rem -= q$ ;
11         if  $rem = 0$  then break;
12     end
13     if  $rem > 0$  then return neuspešno;
14 end
15 return uspešno

```

2.3.3 Faza konstrukcije – klasa Construction

Kupci se obrađuju u opadajućem redosledu po potražnji. Za svakog kupca k , u svakom koraku se formira RCL od parova (i, j) sa ocenom

$$\text{score}(i, j) = b_{ij} + c_{jk} + \frac{f_{ij}}{\text{need}} \cdot [x_{ij} = 0] + \frac{g_{jk}}{\text{need}} \cdot [y_{jk} = 0], \quad (9)$$

gde je need preostala potražnja kupca k u tekućem koraku, a $[x_{ij} = 0]$ i $[y_{jk} = 0]$ indikatori koji dodaju amortizovani fiksni trošak samo za veze koje još nisu aktivirane. RCL sadrži sve parove čija je ocena $\leq \text{score}_{\min} + \alpha(\text{score}_{\max} - \text{score}_{\min})$, a iz njega se nasumično bira par kome se dodeljuje tok $q = \min(\text{rem_s}_i, \text{need})$.

Diversifikacija na nivou skupa otvorenih DC-ova postiže se parametrom `random_dc_mask`: sa verovatnoćom $p = 0,5$, konstrukcija (i naredna lokalna pretraga) ograničava se na nasumično izabran podskup DC-ova čija je veličina između $[0, 35J]$ (najmanje 1) i $J - 1$. Ovim se razbija determinizam pohlepne konstrukcije – bez maske, jeftiniji DC-ovi pobeđuju pri svakom restartu i sva rešenja konvergiraju u isti skup otvorenih DC-ova. Pseudokod je dat Algoritmom ??.

Algoritam 3: GreedyRandomizedConstruction($D, \alpha, \text{allowed_dc}$)

```
1  $sol \leftarrow$  prazno rešenje;  
2 for  $k$  u opadajućem redosledu po  $d_k$  do  
3    $need \leftarrow d_k$ ;  
4   while  $need > 0$  do  
5     Vektorizovano izračunaj  $I \times J$  matricu ocena  $\text{score}(i, j)$ ;  
6     Maskiraj dobavljače sa  $\text{rem\_s}[i] = 0$  i DC-ove van  $\text{allowed\_dc}$  (ocena  
        $= \infty$ );  
7      $\text{score}_{\min} \leftarrow \min$ ,  $\text{score}_{\max} \leftarrow \max$  (nad konačnim vrednostima);  
8      $\text{RCL} \leftarrow \{(i, j) : \text{score}(i, j) \leq \text{score}_{\min} + \alpha(\text{score}_{\max} - \text{score}_{\min})\}$ ;  
9     Nasumično izaberi  $(i^*, j^*)$  iz RCL;  
10     $q \leftarrow \min(\text{rem\_s}[i^*], need)$ ;  
11     $x_{i^*j^*} += q$ ;  $y_{j^*k} += q$ ;  $\text{rem\_s}[i^*] -= q$ ;  $need -= q$ ;  
12  end  
13 end  
14 Izračunaj cenu  $sol$ ;  
15 return  $sol$ 
```

2.3.4 Lokalna pretraga – klasa LocalSearch (VND)

Primenjuje se *Variable Neighborhood Descent* (VND) sa pet okolina, metodom *first improvement* – prvi pronađeni pomak koji smanjuje cenu prihvata se i pretraga se vraća na prvu okolinu. Ako nijedna okolina ne poboljša rešenje, brojač *no_improve* se uvećava; posle $\text{max_no_improve} = 10$ neuspešnih ciklusa pretraga se zaustavlja.

1. **Preraspoređivanje kupca** (*_reassign_customer*): ovaj potez pokušava da kupca k , koga trenutno može opsluživati više DC-ova, prebaci u celosti na jedan DC j_{to} . Prvo se DC-ovi poređaju po tome koliko bi koštalo opsluživanje tog kupca sa svakog od njih (trošak II. etape); zatim se od najviše 8 njih (najboljih 6 plus 2 nasumična, radi istraživanja) redom proba svaki, i prihvata se prvi koji stvarno smanji ukupnu cenu rešenja. Rangiranje po trošku II. etape samo bira redosled isprobavanja – ne odbacuje kandidate unapred, jer ušteda ponekad dolazi iz I. etape, a ne iz II.
2. **Dodavanje DC-a** (*_add_dc*): ovaj potez pokušava da otvori jedan zatvoreni DC; od svih trenutno zatvorenih DC-ova isprobava se najviše 36 kao kandidati. Za razliku od prethodnog poteza, ovde se ne premešta ceo kupac odjednom, već pojedinačne veze (j, k) – tako kupac koji se već deli između više DC-ova zadržava ostatak svoje podele, a menja se samo jedna njegova veza. Za svaki kandidatski DC j_{new} prvo se za svaku postojeću vezu izračuna koliko bi (u trošku II. etape) koštalo da se ta veza umesto na svoj trenutni DC preusmeri na j_{new} , a zatim se probaju dve varijante otvaranja tog DC-a (prva koja da jeftinije rešenje se prihvata):
 - (i) preusmere se *sve* veze kod kojih je to preusmeravanje samo po sebi jeftinije u II. etapi;
 - (ii) preusmere se samo 3 veze kod kojih je to preusmeravanje najmanje skuplje (bez obzira da li je uopšte jeftinije) – ova eksplorativna varijanta otvara DC i kada II. etapa sama po sebi ne opravdava potez, jer se ušteda može javiti tek kroz I. etapu.

3. **Uklanjanje DC-a** (`_remove_dc`): isprobava se do `REMOVE_MAX_CANDIDATES = 20` otvorenih DC-ova za zatvaranje; za svakog pogođenog kupca vektorizovano se bira najjeftinija preostala alternativa.
4. **Zamena DC-a** (`_swap_dc`): jedan otvoreni DC se zamenjuje jednim zatvorenim; oslobođeni kupci se preraspodeljuju na novootvoreni ili na najjeftiniji od preostalih otvorenih DC-ova (videti i odeljak ??).
5. **Pomeranje dela toka** (`_shift_flow`): dok prethodni potezi uvek premeštaju *celu* vezu ili celog kupca, ovaj potez sme da premesti samo *deo* količine sa postojeće veze između DC-a j i kupca k na drugi, ciljni DC j_{to} , ostavljajući ostatak na j – kupac k posle toga ima dve veze umesto jedne, tj. postaje podeljen. To je jedina okolina koja ovako deli kupca; sve ostale ili premeštaju čitavu vezu ili je ostavljaju netaknutom. Za količinu koja se premešta isprobavaju se četiri opcije: cela veza, polovina veze, ili jedna od dve količine izračunate tako da posle premeštanja protok kroz izvorni odnosno ciljni DC postane umnožak kapaciteta jednog dobavljača – to poslednje je bitno jer dobavljači snabdevaju u celim komadima kapaciteta, pa DC čiji je protok tačan umnožak kapaciteta može u potpunosti osloboditi jednog (najskupljeg) dobavljača u I. etapi. Potez uzorkuje najviše 200 veza, za svaku računa tačnu promenu troška II. etape za sve četiri opcije, i redom (od najjeftinije ka najskupljijoj) proba par (veza, količina, ciljni DC) dok neki ne smanji ukupnu cenu. Deljenje kupca samo po sebi nikad ne smanjuje trošak II. etape (koji je linearan po količini plus fiksna doplata), pa se ovaj potez isplati jedino kroz I. etapu – svaki dobavljač ima ograničen kapacitet, pa kada se njegovi najjeftiniji dobavljači potroše, dalje snabdevanje jednog DC-a postaje sve skuplje; deljenje kupca na dva DC-a može tada biti jeftinije nego snabdevati ga u celosti sa jednog.

Razlog za dodatni korak je sama procedura REPAIR: ona obrađuje DC-ove redom po indeksu, pa DC 1 uvek prvi bira najjeftinije dobavljače, a DC-ovi koji dođu na red kasnije nasleđuju ono što je preostalo – rezultat nije nužno ni lokalno optimalan u I. etapi. Na konvergenciji VND ciklusa zato se poziva procedura `polish_x`, koja uz fiksirano y dodatno poboljšava trošak I. etape. Ona radi isključivo u matrici x : bira dva postojeća luka (i_1, j_1) i (i_2, j_2) sa različitim dobavljačima i različitim DC-ovima, i razmenjuje po njima jednaku količinu d unakrsno – $x_{i_1 j_1}$ i $x_{i_2 j_2}$ se umanje za d , a $x_{i_1 j_2}$ i $x_{i_2 j_1}$ uvećaju za d (tzv. četvorougona razmena). Ovakva razmena ne menja ni koliko svaki dobavljač ukupno isporučuje ni koliko svaki DC ukupno prima, pa y i dopustivost rešenja ostaju netaknuti – menja se samo *koji* dobavljač snabdeva koji DC. Za svaki par lukova unapred se, bez pravljenja probnog rešenja, izračuna tačna promena troška takve razmene, i uvek se bira par sa najvećom uštedom (best-improvement), sve dok postoji razmena koja smanjuje trošak ili dok se ne dostigne najviše 200 pokušaja. Pošto broj korišćenih lukova raste sa veličinom instance, kandidati se uzorkuju ako ih ima više od $\max_arcs = \max(600, 2(I + J))$. Pseudokod je dat Algoritmom ??.

Algoritam 4: VND($D, sol, allowed_dc, max_no_improve = 10$)

```
1 REPAIR( $sol, D$ ); Izračunaj cenu  $sol$ ;  
2  $no\_improve \leftarrow 0$ ;  
3 while  $no\_improve < max\_no\_improve$  do  
4   if  $ReassignCustomer(sol)$  then  
5      $no\_improve \leftarrow 0$ ; continue  
6   else if  $AddDC(sol)$  then  
7      $no\_improve \leftarrow 0$ ; continue  
8   else if  $RemoveDC(sol)$  then  
9      $no\_improve \leftarrow 0$ ; continue  
10  else if  $SwapDC(sol)$  then  
11     $no\_improve \leftarrow 0$ ; continue  
12  else if  $ShiftFlow(sol)$  then  
13     $no\_improve \leftarrow 0$ ; continue  
14  else  
15     $no\_improve += 1$   
16  end  
17 end  
18  $polish\_x(sol, D)$ ;  
19 return  $sol$ 
```

2.3.5 Skup elitnih rešenja – klasa ElitePool

Čuva se skup od do 12 najkvalitetnijih i međusobno raznolikih rešenja. Raznolikost se meri Jaccardovim rastojanjem između skupova otvorenih DC-ova dva rešenja a i b :

$$d(a, b) = 1 - \frac{|a.open_dc \cap b.open_dc|}{|a.open_dc \cup b.open_dc|}, \quad (10)$$

sa pragom $min_dist = 0,2$. Pravilo ažuriranja prati standardnu politiku elitnog skupa: dok skup nije pun, odbijaju se samo rešenja sa istim skupom otvorenih DC-ova (ukoliko nisu jeftinija od tog člana); kada je pun, kandidat koji je suviše blizu nekom članu ($open_dc$ rastojanje $\leq min_dist$) takmiči se samo sa *tim* najbližim članom i može ga zameniti jedino ako ima nižu cenu; kandidat dovoljno udaljen od svih članova zamenjuje najskupljeg člana ako je jeftiniji od njega. Time jeftino rešenje nikad ne može da istisne raznolikost skupa – može da zameni samo sopstvenog najbližeg suseda. Metode `best()` i `random()` vraćaju najjeftinije, odnosno nasumično rešenje iz skupa, i koriste se kao ulaz za prekrajanje putanja.

2.3.6 Prekrajanje putanja – klasa PathRelinking

Svakih 8 iteracija uzimaju se dva rešenja iz skupa elitnih rešenja – zovimo ih a (najbolje) i b (nasumično izabrano) – a između njih se izvodi *prekrajanje putanja* razmenom kolona kupaca. Ključno zapažanje: u svakom dopustivom rešenju kolona $y[:, k]$ sabira tačno d_k , pa zamena kolone jednog kupca u rešenju a kolonom tog istog kupca iz rešenja b zadržava II. etapu tačno dopustivom – treba samo ponovo izgraditi I. etapu procedurom REPAIR.

Postupak polazi od a i, kolonu po kolonu, pretvara ga u b : neka je *cur* tekuće (menjano) rešenje, koje na početku jednako a , a b nepromenjeni cilj kome *cur* teži. U svakom koraku

se od kupaca čije se kolone u cur i b još razlikuju nasumično izabere do $\max_cand = 8$ njih; za svakog od njih se probno napravi kopija cur u kojoj je njegova kolona zamenjena odgovarajućom kolonom iz b , i izračuna cena te kopije posle REPAIR-a. Od tih probnih zamena, u cur se stvarno primeni ona sa najnižom cenom – bez obzira da li je ta cena niža ili viša od cene trenutnog cur pre zamene. Postupak, drugim rečima, ne prihvata samo poboljšanja: on nastavlja i kad ga sledeći korak privremeno poskupi, jer je cilj da cur stigne do b , a ne da se zaustavi na prvom lokalnom minimumu. Zato se posebno pamti *najjeftinije* rešenje viđeno duž celog postupka – ono se na kraju vraća kao rezultat, bez obzira gde se postupak zaustavio. Postupak se zaustavlja posle najviše $\max_steps = 15$ koraka ili kada cur postane jednako b (sve kolone usaglašene).

Ovo se radi u *oba* smera – jednom kao $a \rightarrow b$ (cur kreće od a i cilja b), jednom kao $b \rightarrow a$ (cur kreće od b i cilja a) – pošto su međurešenja na ta dva puta različita; čuva se najjeftinija tačka viđena na bilo kojem od njih, i ona se na kraju finalizuje lokalnom pretragom (VND). Skupovi otvorenih DC-ova ne igraju ulogu u postupku – unija i presek otvorenih DC-ova roditelja ne koriste se, već se putanja definiše isključivo razlikama u kolonama kupaca. Pseudokod je dat Algoritmom ??.

Algoritam 5: PathRelinking(a, b, D)

```

1 best  $\leftarrow \emptyset$ ; best_cost  $\leftarrow +\infty$ ;
2 for ( $src, dst$ )  $\in \{(a, b), (b, a)\}$  do
3    $cur \leftarrow src.copy()$ ;
4    $diff \leftarrow \{k : cur.y[:, k] \neq dst.y[:, k]\}$ ;
5   steps  $\leftarrow 0$ ;
6   while  $diff \neq \emptyset$  i steps  $< \max\_steps$  do
7     cands  $\leftarrow$  nasumični podskup od  $\leq \max\_cand$  kupaca iz  $diff$ ;
8      $k^* \leftarrow \arg \min_{k \in cands} \text{cost}(\text{Repair}(cur, D) \text{ posle } cur.y[:, k] \leftarrow dst.y[:, k])$ ;
9      $cur.y[:, k^*] \leftarrow dst.y[:, k^*]$ ;
10    REPAIR( $cur, D$ ); izračunaj cenu  $cur$ ;
11     $diff \leftarrow diff \setminus \{k^*\}$ ; steps  $+= 1$ ;
12    if  $cur.cost < best\_cost$  then
13      | best  $\leftarrow cur.copy()$ ; best_cost  $\leftarrow cur.cost$ ;
14    end
15  end
16 end
17 return VND( $D, best$ )

```

U paralelnoj verziji prekrajanje putanja se izvršava u radnicima, a ne u glavnom procesu (videti odeljak ??): merenja su pokazala da iznosi oko 30% ukupnog vremena izvršavanja, pa bi sekvencijalno izvršavanje držalo sve radnike neaktivnima dok glavni proces ne završi.

2.3.7 Perturbacija i intenzifikacija

Perturbacija se koristi na dva nezavisna nivoa: ILS perturbacija je *intenzifikacioni* mehanizam (ponovni spust elitnog rešenja u njegovom DC-okruženju), dok je ruin-and-recreate perturbacija *diversifikacioni* mehanizam koji se aktivira tek pri dugotrajnom zastoju.

- **ILS perturbacija:** sa verovatnoćom $p_{seed} = 0,5$ iteracija ne kreće iz praznog rešenja, već se nasumično odabrano elitno rešenje (*seed*) perturbuje – nasumičan broj otvorenih DC-ova (do otprilike trećine) se zatvori, a njihovi lukovi se *nasumično*

preliju na druge DC-ove (namerno ne pohlepno, jer bi pohlepno presmeravanje bilo samo još jedan lokalni potez koji bi VND odmah poništio) – a zatim se spušta lokalnom pretragom. Ponovni spust se ograničava maskom `elite_dc_mask`, koja lokalnu pretragu zadržava na skupu otvorenih DC-ova semena uvećanom za $n_{extra} = 1$ nasumično izabrani zatvoreni DC. Bez maske bi se svaka perturbacija elitnog rešenja vratila u isti generički skup otvorenih DC-ova, pa skup DC-ova koji prekrajanje putanja pokušava da pronađe nikad ne bi bio intenzifikovan.

- **Ruin-and-recreate perturbacija:** ako se 20 uzastopnih iteracija ne pronađe poboljšanje najboljeg rešenja, DC-ovi se rangiraju po prosečnom varijabilnom trošku po jedinici protoka, uklanja se 50% najskupljih, a kupci koji su im bili dodeljeni presmeravaju se na preostale DC-ove po najnižoj oceni troška. Posle REPAIR-a i provere dopustivosti primenjuje se VND; poboljšanje se prihvata kao novo najbolje, a i samo rešenje se, kao potpuno spuštено, dodaje u skup elitnih rešenja bez obzira na poboljšanje. Brojač zastoja se resetuje posle svakog pokušaja (bez obzira na uspeh), čime se sprečavaju uzastopne perturbacije iz istog stanja.

2.3.8 Adaptivno podešavanje parametra α

Ako je prošlo više od 15 iteracija bez poboljšanja najboljeg rešenja, α se uzorkuje iz $\mathcal{U}(0,4, 0,7)$ radi diversifikacije pretrage (veći RCL, više nasumičnosti). Inače, $\alpha \in \mathcal{U}(0,05, 0,5)$ radi intenzifikacije (manji RCL, bliže pohlepnom izboru). Reč je o *reaktivnom podešavanju zasnovanom na stagnaciji*, a ne o klasičnom Reactive GRASP-u, u kome se verovatnoće pojedinih vrednosti α prilagođavaju prema kvalitetu rešenja koja su proizvele; ovde se α bira iz dva fiksna opsega, u zavisnosti od toga da li pretraga napreduje.

2.3.9 Egzaktno filtriranje kandidata

Puna evaluacija jednog kandidata je skupa: treba napraviti probno rešenje, pozvati REPAIR (koji je $O(I \times J)$) i tek onda izračunati cenu. Kad okolina ima na desetine ili stotine kandidata po pozivu, proveru svih na ovaj način je najskuplji deo pretrage. Zato se u više okolina (`_swap_dc`, `_shift_flow`, `_reassign_customer`) koristi **dvostepena evaluacija**: prvo se svi kandidati jeftino poređaju po *tačnoj* promeni troška II. etape – koja se, za razliku od troška I. etape, može izračunati direktno iz formule, bez pravljenja probnog rešenja i bez REPAIR-a – a zatim se REPAIR i puna cena računaju samo za mali broj najbolje rangiranih kandidata, plus nekoliko nasumičnih. Ovi nasumični kandidati su tu jer rangiranje vidi samo II. etapu: kandidat koji izgleda loše u II. etapi može ipak biti dobar zbog uštede u I. etapi (npr. rasterećenja skupog dobavljača), pa rangiranje sme da bira *redosled* provere, ali ne sme unapred da odbaci kandidate.

Ilustracije radi, posmatrajmo kako to radi okolina zamene DC-a (`_swap_dc`): jedan otvoreni DC j' se zatvara, a jedan zatvoreni j'' otvara, pa svi kupci koje je opsluživao j' moraju da se presele na neki drugi otvoreni DC ili na j'' . Za dati par (j', j'') , trošak II. etape te zamene se može izračunati unapred, bez ikakve simulacije: za svakog kupca k koga je opsluživao j' dovoljno je znati koliko bi koštalo da ga umesto toga opslužuje bilo koji drugi trenutno otvoren DC ili j'' (trošak = varijabilni trošak c_{jk} puta količina, plus fiksni trošak g_{jk} ako ta veza još nije u upotrebi), i uzeti najjeftiniju opciju za tog kupca. Zbir tih najjeftinijih opcija po svim oslobođenim kupcima je egzaktan trošak II. etape cele zamene – razlika (Δ) tog troška u odnosu na sadašnji trošak istih kupaca je ključ po kome se par (j', j'') rangira. Formalno, uz $q_{j'k} = y_{j'k}$ (koliko kupac k trenutno dobija od

j') i $K_{j'}$ (skup kupaca koje opslužuje j'), trošak II. etape opsluživanja kupca k iz DC-a j je

$$s_2(j, k) = c_{jk} q_{j'k} + g_{jk} [y_{jk} = 0], \quad (11)$$

gde indikator $[y_{jk} = 0]$ dodaje fiksni trošak samo ako ciljni luk nije već u upotrebi, pa je egzaktni trošak II. etape cele zamene

$$C_{j',j''}^{(2)} = \sum_{k \in K_{j'}} \min \left(s_2(j'', k), \min_{j \in J_{\text{open}} \setminus \{j'\}} s_2(j, k) \right). \quad (12)$$

Svi parovi (otvoren, zatvoren) rangiraju se po $\Delta = C_{j',j''}^{(2)} - C_{j'}^{(2)}$, gde je $C_{j'}^{(2)}$ tekući trošak istih kupaca dok ih još opslužuje j' . Ovo se ne mora ponavljati za svaki par posebno: oslobođeni kupci zavise samo od j' , pa se izračunavanje grupiše po j' , i cela $|J_{\text{open}}| \times |J_{\text{closed}}|$ mreža parova košta samo $|J_{\text{open}}|$ vektorizovanih prolaza. U punoj evaluaciji (probno rešenje, REPAIR, prava cena) izvršavaju se samo najbolje rangiranih $\text{SWAP_MAX_TRIALS} = 12$ parova, plus $\text{SWAP_EXPLORE} = 2$ nasumično izabrana iz ostatka od $\text{SWAP_MAX_PAIRS} = 150$ rangiranih.

Isto načelo primenjuje se i u ostalim okolinama, sa budžetima definisanim kao atributi klase (npr. $\text{SHIFT_MAX_ARCS} = 200$, $\text{SHIFT_MAX_TRIALS} = 40$ za okolinu pomeranja dela toka). Merenja pokazuju da filtriranje ne pogoršava kvalitet rešenja, a ubrzava lokalnu pretragu za 10–20% na instancama gde je budžet aktivan.

2.3.10 Vektorizacija i paralelizacija

Svi najzahtevniji delovi algoritma – izračunavanje matrica ocena u konstrukciji, evaluacija kandidata u okolinama i `compute_cost()` – vektorizovani su preko NumPy broadcast-a i boolean maski, bez promene rezultata pretrage (ista RCL/ α logika i ista *first-improvement* pravila, samo izračunata bržim putem).

GRASP petlja je inherentno sekvencijalna (skup elitnih rešenja, praćenje najboljeg rešenja, prekrajanje putanja i perturbacija zavise od deljenog stanja), ali konstrukcija i lokalna pretraga jedne iteracije zavise samo od parametra α (odnosno odabranog semena za ILS granu) i od nepromenjenih podataka instance. Zato se batch od n_{workers} nezavisnih zadataka izvršava paralelno preko `multiprocessing.Pool`:

- Podaci instance šalju se svakom radniku tačno jednom, preko `initializer` funkcije; radnici vraćaju samo rezultat (x, y , cena, `open_dc`, broj poziva funkcije cilja).
- Svaki zadatak je *tagovana* n-torka: (`'ls'`, α , `None`) – običan GRASP restart (konstrukcija + VND pod nasumičnom maskom DC-ova); (`'ls'`, α , `seed_y`) – ILS iteracija (perturbacija elitnog rešenja i ponovni spust pod `elite_dc_mask`); ili (`'pr'`, ...) – prekrajanje putanja između dva elitna rešenja.
- Sekvencijalni deo – ažuriranje skupa elitnih rešenja, praćenje najboljeg rešenja, prekrajanje putanja i perturbacija – primenjuje se na rezultate iz batch-a istim redosledom kao u serijskoj verziji. To nije bit-identično serijskoj petlji: parametar α , ILS semena i roditelji za prekrajanje putanja fiksirani su na *početku* batch-a, pa konstrukcija+VND (odnosno prekrajanje putanja) rade nad stanjem skupa elitnih rešenja kakvo je bilo pre nego što je batch započeo, a ne nad stanjem koje bi u serijskom režimu već apsorbovalo ranije iteracije batch-a.
- Ako je $n_{\text{workers}} \leq 1$, koristi se u potpunosti originalna serijska petlja – ponašanje je tada identično algoritmu bez paralelizacije.

Broj radnika se podrazumevano bira kao `cpu_count() - 1`, a može se eksplicitno podesiti preko CLI opcije `-workers`. Pseudokod paralelne verzije dat je Algoritmom ??.

Algoritam 6: `GRASP_Parallel( $D, T, n_{\text{workers}}$ )` – razlika u odnosu na serijsku verziju

```

1 Pokreni Pool od  $n_{\text{workers}}$  procesa, pošalji im  $D$  jednom;
2  $it \leftarrow 0$ ;
3 while  $it < T$  do
4      $batch \leftarrow \min(n_{\text{workers}}, T - it)$ ;
5     Generiši batch zadataka: konstrukcija+VND (svež restart ili ILS), a svakih 8
        iteracija i prekrajanje putanja između dva elitna rešenja;
        //  $\alpha$ , ILS semena i roditelji za PR uzimaju se iz stanja na
        početku batch-a
6      $results \leftarrow$  paralelno izračunaj sve zadatke iz batch-a;
7     for svaki rezultat iz results (sekvencijalno, istim redosledom kao serijska
        GRASP petlja) do
8         Ažuriraj skup elitnih rešenja, najbolje rešenje, (svakih 8) prekrajanje
            putanja, (posle 20 zastoja) perturbaciju;
9          $it += 1$ ;
10    end
11 end
12 Zatvori Pool;
13 return najbolje pronađeno rešenje

```

2.3.11 Kompletan pseudokod GRASP algoritma

Kompletan pseudokod (serijski ekvivalent) dat je Algoritmom ???. Paralelna verzija (Algoritam ??) izvršava telo petlje po it na isti način po *redosledu* sekvencijalnog dela, s tim što se konstrukcija+VND (odnosno prekrajanje putanja) za grupu od n_{workers} uzastopnih vrednosti it računaju paralelno pre nego što se sekvencijalni deo (ažuriranje skupa elitnih rešenja, najboljeg rešenja, prekrajanja putanja i perturbacije) primeni na svaki rezultat iz batch-a, tim redosledom – pri čemu parametar α , ILS semena i roditelji za prekrajanje putanja odgovaraju stanju na početku batch-a, a ne stanju koje bi serijska petlja imala nakon prethodnih iteracija unutar batch-a (videti odeljak ??).

Algoritam 7: GRASP(D, T)

Input: Podaci instance D , broj iteracija T **Output:** Najbolje pronađeno rešenje sol^*

```
1  $sol^* \leftarrow \emptyset$ ;  $best\_cost \leftarrow +\infty$ ;  $last\_imp \leftarrow 0$ ;  $E \leftarrow \emptyset$  (skup elitnih rešenja,
   veličina 12);
2 for  $it = 0$  to  $T - 1$  do
3   if  $it - last\_imp > 15$  then
4      $\alpha \leftarrow \mathcal{U}(0,4, 0,7)$ ;
5   else
6      $\alpha \leftarrow \mathcal{U}(0,05, 0,5)$ ;
7   end
8   if  $|E| \geq 1$  i nasumično sa verovatnoćom  $p_{seed}$  then
9      $seed \leftarrow E.random()$ ;
10     $mask \leftarrow elite\_dc\_mask(seed)$ ; // iz neperturbovanog semena
11     $sol \leftarrow Perturbacija(seed, D)$ ;
12     $sol \leftarrow VND(D, sol, mask)$ ;
13  else
14     $mask \leftarrow RandomDcMask(D.J)$ ;
15     $sol \leftarrow GreedyRandomizedConstruction(D, \alpha, mask)$ ;
16     $sol \leftarrow VND(D, sol, mask)$ ;
17  end
18   $E.add(sol)$ ;
19  if  $sol.cost < best\_cost$  then
20     $sol^* \leftarrow sol$ ;  $best\_cost \leftarrow sol.cost$ ;  $last\_imp \leftarrow it$ ;
21  end
22  if  $it \bmod 8 = 0$  i  $|E| \geq 2$  then
23     $cand \leftarrow PathRelinking(E.best(), E.random(), D)$ ;
24     $E.add(cand)$ ;
25    if  $cand.cost < best\_cost$  then
26       $sol^* \leftarrow cand$ ;  $best\_cost \leftarrow cand.cost$ ;  $last\_imp \leftarrow it$ ;
27    end
28  end
29  if  $it - last\_imp > 20$  then
30     $sol' \leftarrow RuinAndRecreate(sol^*, D)$ ;
31    if  $REPAIR(sol', D)$  i  $sol'.is\_valid()$  then
32       $sol' \leftarrow VND(D, sol')$ ;
33       $E.add(sol')$ ;
34      if  $sol'.cost < best\_cost$  then
35         $sol^* \leftarrow sol'$ ;  $best\_cost \leftarrow sol'.cost$ ;  $last\_imp \leftarrow it$ ;
36      end
37    end
38     $last\_imp \leftarrow it$ ; // sprečava uzastopne perturbacije
39  end
40 end
41 return  $sol^*$ 
```

3 Rezultati testiranja i analiza

3.1 Test instance

Za testiranje je korišćen skup od 60 instanci podeljenih u tri grupe (generisane koordinatnim modelom – čvorovi nasumično u kvadratu $[-400, 400]^2$, varijabilni troškovi su Euklidske distance, fiksni troškovi su višekratnici tih distanci, $r_f \in [10, 15]$ za I. etapu i $r_g \in [5, 10]$ za II. etapu):

- **Male** (small, 20 instanci): $I \in [4, 18]$, $J \in [8, 36]$, $K \in [16, 80]$.
- **Srednje** (medium, 20 instanci): $I \in [20, 60]$, $J \in [40, 120]$, $K \in [80, 275]$.
- **Velike** (large, 20 instanci): $I \in [70, 160]$, $J \in [140, 320]$, $K \in [280, 800]$.

Svaki dobavljač ima kapacitet $s_i = 80$; potražnja kupaca $d_k \in [10, 30]$ je generisana nasumično, a zatim normalizovana tako da ukupna potražnja tačno odgovara ukupnoj ponudi ($\sum_k d_k = \sum_i s_i$). U trenutku pisanja rada izvršene su sve male i srednje instance, a od velikih 11 od 20 (`large_1–large_11`); preostalih 9 velikih instanci je u toku i nije uključeno u proseke i analizu.

3.2 Referentna rešenja (CPLEX)

Referentna rešenja dobijena su CPLEX solverom (IBM ILOG CPLEX) korišćenjem MIP formulacije iz odeljka 1, sa vremenskim limitom od 7200 s i jednom niti. Status `OPTI` označava da je CPLEX dokazao optimalnost; status `FEAS` da je dostignut vremenski limit i vraćeno najbolje pronađeno dopustivo rešenje (bez dokaza optimalnosti) – to je i dalje referentna vrednost korišćena za *Bestsol* (optimalno ili najbolje poznato rešenje). Od malih instanci 14 je rešeno optimalno, a 6 uz status `FEAS`; sve srednje i velike instance imaju status `FEAS`.

3.3 Podešavanja GRASP-a, platforma i metodologija izveštaja

Platforma za GRASP: x86_64, 16 CPU jezgara, Python 3.12.3, NumPy 2.4.4. Algoritam je izvršavan paralelno sa $n_{\text{workers}} = \text{cpu_count}() - 1 = 15$ radnika.

Broj izvršavanja i iteracija zavisi od grupe instanci:

- Male instance: $k = 10$ izvršavanja po instanci, $T = 1000$ iteracija;
- Srednje instance: $k = 10$ izvršavanja po instanci, $T = 1000$ iteracija;
- Velike instance: $k = 3$ izvršavanja po instanci, $T = 100$ iteracija.

Označimo sa D_i instancu, a sa r -to izvršavanje ($r = 0, 1, \dots, k - 1$) određeno seed-om $42 + 100r$ (deterministički deo pretrage). Za svako izvršavanje i algoritam beleži izveštajne veličine:

- sol_i – cena najboljeg pronađenog rešenja;
- t_i – vreme od početka izvršavanja do trenutka prvog nalaska sol_i (beleži se u glavnoj petlji, u prekrajanju putanja i u perturbaciji, u tačnom trenutku kada se najbolje rešenje poboljša);
- tot_i – ukupno vreme izvršavanja;

- $eval_i$ – broj poziva funkcije cilja (`eval_cost()`, koji se poziva i iz `compute_cost()` i direktno iz prihvatne probe lokalne pretrage), odnosno globalni brojač koji se uvećava pri svakom stvarnom izračunavanju vrednosti funkcije cilja; u paralelnom režimu svaki radnik ima sopstveni brojač koji se resetuje na početku svakog zadatka i vraća kao deo rezultata, a glavni proces ga sabira sa sopstvenim pozivima.

$Bestsol$ se određuje kao CPLEX referentna vrednost ako je poznata i bolja od najboljeg MA rezultata, u suprotnom kao najbolji MA rezultat kroz k izvršavanja; zatim se za svako izvršavanje računa

$$gap_i = 100 \cdot \frac{|sol_i - Bestsol|}{|Bestsol|}.$$

U tabelama su prikazane srednje vrednosti $\bar{t}$, $\overline{ttot}$, $\overline{eval}$ i $\overline{gap}$, kao i standardna devijacija σ (u % gap-a), računate preko k izvršavanja po instanci.

3.4 Male instance

Tabela 1: Rezultati za male instance ($T = 1000$, $k = 10$)

#	$I/J/K$	Ref. (CPLEX)	Best MA	$\bar{t}$ (s)	$\overline{ttot}$ (s)	$\overline{eval} (\times 10^3)$	$\overline{gap}$ (%)	σ (%)
1	4/8/16	150 568 (OPTI)	150 568	1.4	16.0	1270.2	0.00	0.00
2	4/8/20	156 022 (OPTI)	159 520	3.2	18.1	1472.7	2.24	0.00
3	4/10/20	161 224 (OPTI)	163 375	4.3	24.3	1661.5	1.33	0.00
4	6/12/24	186 537 (OPTI)	187 642	5.0	29.5	1832.8	1.51	0.71
5	6/12/30	213 785 (OPTI)	215 740	20.6	39.7	2351.2	2.13	0.61
6	6/15/30	188 954 (OPTI)	194 359	20.7	48.0	2652.5	3.52	0.61
7	8/16/32	231 984 (OPTI)	238 341	11.4	44.2	2478.8	3.30	0.33
8	8/16/40	202 467 (OPTI)	210 295	43.0	84.1	3223.0	4.61	0.50
9	8/20/40	210 740 (OPTI)	214 410	28.4	87.7	3584.0	1.84	0.29
10	10/20/40	258 236 (OPTI)	264 356	41.2	74.2	3457.6	3.22	0.66
11	10/20/50	286 991 (OPTI)	295 282	51.3	107.0	4412.0	3.48	0.44
12	10/25/50	284 643 (FEAS)	297 645	86.3	144.9	5795.7	4.87	0.21
13	12/24/48	278 044 (OPTI)	290 923	99.2	134.8	4898.2	4.98	0.39
14	12/24/60	284 648 (OPTI)	304 953	79.1	141.2	6274.8	7.66	0.39
15	12/30/60	315 417 (OPTI)	328 859	124.0	180.4	6230.3	4.88	0.43
16	14/28/56	349 820 (FEAS)	363 512	151.9	207.7	6465.9	4.73	0.46
17	14/28/70	368 942 (FEAS)	377 343	127.0	244.8	8012.0	3.17	0.49
18	16/32/64	349 036 (FEAS)	362 127	112.7	241.4	7093.1	4.38	0.28
19	16/32/80	351 345 (FEAS)	372 703	248.5	319.4	8372.6	6.72	0.39
20	18/36/72	400 600 (FEAS)	420 027	216.7	306.0	7708.9	5.97	0.55
Prosek				73.8	124.7	4462.4	3.73	0.39

3.5 Srednje instance

Tabela 2: Rezultati za srednje instance ($T = 1000$, $k = 10$)

#	$I/J/K$	Ref. (CPLEX)	Best MA	$\bar{t}$ (s)	$\overline{ttot}$ (s)	$\overline{eval} (\times 10^3)$	$\overline{gap}$ (%)	σ (%)	
1	20/40/80	383 558 (FEAS)	398 577	275.8	372.2	9327.8	4.52	0.44	
2	20/40/100	415 983 (FEAS)	434 525	369.3	463.4	12731.1	5.16	0.57	
3	20/50/100	400 875 (FEAS)	428 213	347.5	565.3	13443.9	8.17	0.65	
4	25/50/100	395 660 (FEAS)	414 693	505.3	625.2	13761.7	5.82	0.46	
5	25/50/125	474 978 (FEAS)	504 505	636.6	788.2	16568.3	7.33	0.72	
6	25/60/120	447 777 (FEAS)	474 854	890.2	969.3	19571.9	7.11	0.76	
7	30/60/120	474 863 (FEAS)	498 115	667.6	862.1	15725.4	5.93	0.52	
8	30/60/150	514 195 (FEAS)	553 998	1796.2	2103.5	27681.7	8.87	0.95	
9	30/75/150	503 494 (FEAS)	530 874	1520.2	1800.4	27182.2	6.51	0.59	
10	35/70/140	525 261 (FEAS)	563 494	2024.9	2139.8	25283.5	7.78	0.30	
11	35/70/175	551 427 (FEAS)	576 803	1984.1	2144.5	30091.8	5.98	0.88	
12	40/80/160	478 570 (FEAS)	503 072	2169.6	2519.7	26760.4	6.25	0.77	
13	40/80/200	616 913 (FEAS)	669 048	3196.6	3527.8	41645.2	9.73	0.76	
14	45/90/180	689 812 (FEAS)	741 059	3013.7	3846.8	40371.7	8.13	0.56	
15	45/90/225	557 928 (FEAS)	604 169	3068.2	3942.8	42065.7	8.91	0.44	
16	50/100/200	592 227 (FEAS)	648 696	3706.4	4128.7	39819.4	10.64	0.64	
17	50/100/250	612 725 (FEAS)	652 416	5573.9	6418.6	59909.0	7.71	0.65	
18	55/110/220	685 116 (FEAS)	753 348	5187.0	5969.0	52562.4	11.27	0.67	
19	55/110/275	580 226 (FEAS)	626 831	6643.6	7732.8	64967.1	8.93	0.58	
20	60/120/240	677 683 (FEAS)	729 939	6121.0	6861.8	52925.8	8.59	0.44	
Prosek				2484.9	2889.1	31619.8	7.67	0.62	

3.6 Velike instance

Velike instance predstavljaju najzahtevniju grupu; u trenutku pisanja rada izvršeno je 11 od 20 instanci (`large_1`–`large_11`) sa $k = 3$ izvršavanja po instanci i $T = 100$ iteracija. Preostalih 9 instanci je u toku i nije uključeno u proseke i analizu.

Tabela 3: Rezultati za velike instance ($T = 100$, $k = 3$)

#	$I/J/K$	Ref. (CPLEX)	Best MA	$\bar{t}$ (s)	$\overline{ttot}$ (s)	$\overline{eval} (\times 10^3)$	$\overline{gap}$ (%)	σ (%)	
1	70/140/280	741 988 (FEAS)	829 048	1181.7	1220.4	7665.7	12.98	0.92	
2	70/140/350	825 845 (FEAS)	954 277	1623.8	1901.5	10823.2	16.10	0.43	
3	80/160/320	851 981 (FEAS)	999 912	1781.8	2065.9	10358.8	17.97	0.45	
4	80/160/400	807 109 (FEAS)	909 778	2765.4	2889.6	13246.1	12.87	0.17	
5	90/180/360	801 015 (FEAS)	902 199	2408.0	2650.7	11503.0	13.37	0.55	
6	90/180/450	953 624 (FEAS)	1 075 114	3242.3	4157.6	16344.4	13.83	0.78	
7	100/200/400	1 081 573 (FEAS)	1 237 060	4196.2	4437.0	16028.4	15.32	1.06	
8	100/200/500	916 397 (FEAS)	1 066 027	4872.1	6323.7	19903.8	17.63	0.92	
9	110/220/440	871 261 (FEAS)	1 022 419	5708.3	5708.3	16513.4	18.36	1.00	
10	110/220/550	945 683 (FEAS)	1 080 777	7082.7	8055.4	21771.7	15.29	0.93	
11	120/240/480	971 579 (FEAS)	1 112 135	8410.9	8410.9	21585.8	15.55	0.86	

Nastavlja se na sledećoj strani

Tabela 3 – nastavak

#	$I/J/K$	Ref. (CPLEX)	Best MA	$\bar{t}$ (s)	$\overline{t_{tot}}$ (s)	$\overline{eval} (\times 10^3)$	$\overline{gap}$ (%)	σ (%)
				Prosek	3933.9	4347.4	15067.6	15.39
								0.73

3.7 Analiza rezultata

3.7.1 Analiza – male instance

Na malim instancama GRASP postiže prosečno odstupanje od $\overline{gap} = 3,73\%$ ($\sigma = 0,39\%$) u odnosu na CPLEX referentnu vrednost, uz $k = 10$ nezavisna izvršavanja po instanci. Na instanci 1 ($4 \times 8 \times 16$) algoritam u svih 10 izvršavanja pronalazi tačno optimalno rešenje (gap 0,00%), a na instancama 2 i 3 gap je manji od 2,5%. Standardna devijacija je mala (uglavnom ispod 0,7%), što ukazuje na konzistentno ponašanje algoritma za fiksirani budžet od $T = 1000$ iteracija. Najveći gap postiže se na instanci 14 ($12 \times 24 \times 60$, 7,66%), što ukazuje da porast broja kupaca po DC-u otežava lokalnoj pretrazi da u zadatom broju iteracija pronađe globalni optimum. Prosečno vreme do najboljeg rešenja $\bar{t}$ raste sa dimenzijama instance (od 1,4 s do 248,5 s), dok je ukupno vreme $\overline{t_{tot}}$ svugde ispod 320 s.

3.7.2 Analiza – srednje instance

Na srednjim instancama prosečno odstupanje iznosi $\overline{gap} = 7,67\%$ ($\sigma = 0,62\%$). Sve CPLEX referentne vrednosti za srednje instance imaju status **FEAS** – CPLEX je dostigao vremenski limit od 7200 s bez dokaza optimalnosti, pa referentna vrednost nije nužno optimalna, već najbolje pronađeno dopustivo rešenje kroz snažnu *B&B/cutting-plane* pretragu – suštinski drugačiji (i po vremenu mnogo skuplji) mehanizam od GRASP lokalne pretrage sa $T = 1000$ iteracija. Najmanji gap je na instanci 1 ($20 \times 40 \times 80$, 4,52%), a najveći na instanci 18 ($55 \times 110 \times 220$, 11,27%). Vreme izvršavanja raste sa dimenzijama: $\overline{t_{tot}}$ ide od 372 s (instanca 1) do 7733 s (instanca 19, preko 2 sata po izvršavanju), a broj evaluacija funkcije cilja $\overline{eval}$ raste do približno 65 miliona poziva po izvršavanju.

3.7.3 Analiza – velike instance

Na velikim instancama (11 izvršenih) prosečno odstupanje iznosi $\overline{gap} = 15,39\%$ ($\sigma = 0,73\%$), nastavljajući trend rasta uočen kod malih i srednjih grupa ($3,73\% \rightarrow 7,67\% \rightarrow 15,39\%$) – očekivano, budući da je broj iteracija ($T = 100$) najmanji od sve tri grupe, dok je prostor pretrage (I, J, K do 120, 240, 480 među izvršenim instancama) najveći. Najmanji gap je na instanci 4 ($80 \times 160 \times 400$, 12,87%), a najveći na instanci 9 ($110 \times 220 \times 440$, 18,36%); razlike između instanci ukazuju da na ovim dimenzijama varijabilnost zavisi od konkretne strukture instance, ne samo od njene veličine. Vreme izvršavanja raste dramatično: $\overline{t_{tot}}$ ide od 1220 s (instanca 1) do preko 8400 s (instanca 11, oko 2,5 sata po izvršavanju). Očekuje se da preostalih 9 instanci (do $160 \times 320 \times 800$) bude znatno zahtevnije.

3.7.4 Zajednički zaključci

- Odstupanje GRASP-a od CPLEX referentne vrednosti raste sa dimenzijom instance ($3,73\% \rightarrow 7,67\% \rightarrow 15,39\%$), što je očekivana posledica fiksnog broja iteracija po grupi ($1000 \rightarrow 1000 \rightarrow 100$) uz brzo rastući prostor pretrage.

- Standardna devijacija σ je kroz sve tri grupe relativno mala (prosečno 0,39–0,73%, pojedinačno najviše do $\approx 1,1\%$), što pokazuje da je algoritam, uz višestruka izvršavanja po instanci, dovoljno konzistentan – razlike između pojedinačnih izvršavanja iste instance znatno su manje od razlika između instanci.
- Za srednje i velike instance (i deo malih) CPLEX referentne vrednosti same po sebi nisu dokazano optimalne (status **FEAS**), pa je stvarni gap GRASP-a prema (nepoznatom) optimumu verovatno nešto manji od prikazanog.
- Vektorizacija i paralelizacija (odeljci 2.2.9–2.2.10) preduslov su da je $k = 10$ izvršavanja po svih malih i srednjih instanci uopšte bilo izvodljivo u razumnom vremenu; egzaktno filtriranje kandidata dodatno ubrzava lokalnu pretragu bez pogoršanja kvaliteta.

4 Zaključak

U ovom radu predložena je GRASP metaheuristika za Dvostepeni transportni problem sa fiksnim troškovima (TS-FCTP). Algoritam kombinuje pohlepno-nasumičnu konstrukciju sa adaptivno podešavanjem parametrom α , VND lokalnu pretragu sa pet okolina, egzaktno filtriranje kandidata po zatvorenoj formuli za trošak druge etape, skup elitnih rešenja sa raznolikošću po Jaccardovom rastojanju, prekrajanje putanja razmenom kolona kupaca (u paralelnoj verziji izvršavan u radnicima), ILS perturbaciju elitnih rešenja ograničenu maskom na DC konfiguraciju semena i ruin-and-recreate perturbaciju pri dugotrajnom zastoju.

Centralna arhitekturna odluka – direktna pretraga u prostoru y -tokova uz rekonstrukciju x -tokova procedurom REPAIR – efikasno smanjuje dimenzionalnost problema. Nad tom osnovom, implementacija je dodatno vektorizovana (NumPy) i paralelizovana (**multiprocessing**, batch nezavisnih iteracija), što je omogućilo višestruka izvršavanja po instanci i dobijanje statistički smislenih ocena ($\overline{gap}$, σ).

Testiranje pokazuje da GRASP postiže dobra rešenja na malim instancama (prosečan gap 3,73%, $\sigma = 0,39\%$, sa tačno optimalnim rešenjem na najmanjoj instanci), uz umeren pad kvaliteta na srednjim instancama (gap 7,67%, $\sigma = 0,62\%$) i dalji pad na velikim instancama (gap 15,39%, $\sigma = 0,73\%$, na 11 od 20 izvršenih instanci), usled fiksnog i sve manjeg broja iteracija u odnosu na sve veći prostor pretrage na krupnijim problemima. Kroz sve tri grupe standardna devijacija ostaje relativno mala, što ukazuje na konzistentno ponašanje algoritma nezavisno od dimenzije instance.

Mogući pravci daljeg razvoja:

- dovršiti izvršavanje preostalih 9 velikih instanci i uporediti rezultate na kompletnom skupu;
- povećati broj iteracija T na velikim instancama (npr. dodatnim ubrzanjem REPAIR-a) kako bi se smanjio uočeni gap na toj grupi;
- istražiti varijante prekrajanja putanja (npr. duži postupak, izbor koraka prema uticaju na I. etapu) kako bi se međutačke bolje iskoristile na velikim instancama.

Literatura

- [1] N. Jawahar, A. N. Balaji, *A genetic algorithm for the two-stage supply chain distribution problem associated with a fixed charge*, European Journal of Operational Research **194**(2), 496–537 (2009).
- [2] H. I. Calvete, C. Galé, J. A. Iranzo, P. Toth, *A matheuristic for the two-stage fixed-charge transportation problem*, Computers & Operations Research **95**, 113–122 (2018).
- [3] A. N. Balaji, N. Jawahar, *A simulated annealing algorithm for a two-stage fixed charge distribution problem of a supply chain*, International Journal of Operational Research **7**(2), 192–215 (2010).
- [4] K. A. A. D. Raj, C. Rajendran, *A genetic algorithm for solving the fixed-charge transportation model: two-stage problem*, Computers & Operations Research **39**(9), 2016–2032 (2012).
- [5] P. C. Pop, O. Matei, C. Pop Sitar, I. Zelina, *A hybrid based genetic algorithm for solving a capacitated fixed-charge transportation problem*, Carpathian Journal of Mathematics **32**(2), 225–232 (2016).
- [6] T. A. Feo, M. G. C. Resende, *Greedy randomized adaptive search procedures*, Journal of Global Optimization **6**(2), 109–133 (1995).
- [7] B. Eksioglu, A. Migdalas, P. M. Pardalos, *Heuristic approaches to production-inventory-distribution problems in supply chains*, in: Optimization and Industry: New Frontiers, Kluwer Academic Publishers, 15–37 (2003).